

Autonomous Testing

with Vilhelm von Ehrenheim

Donato Giuliano - UO318987

Marvin Kickel - UO318535

Daniel Onís Fabián – UO293946

Agenda

The Evolution & Fundamentals

AI Agents in Action

Implementation and Metrics

PART 1 OF 3

The Evolution & Fundamentals: Autonomous Testing

The "What" and the "Why" — Defining the technology and explaining the relevance

The Testing Evolution

The Power of Self-Healing

Prerequisites for Adoption

The Testing Evolution



Manual Testing

The Approach: 100% human-driven interaction.

The Challenge: Repetitive, time-consuming, and difficult to scale as the application grows.

The Reality: Still valuable today for exploratory testing, but inefficient for continuous regression.



Automated Testing

The Approach: Hardcoded scripts (e.g., Selenium, Cypress) executing pre-defined steps.

The Challenge: Highly fragile and maintenance-heavy. If a button moves or changes color, the test breaks.

The Reality: Requires constant script updates from engineers to keep pace with UI changes.



Autonomous Testing

The Approach: AI-driven agents that understand context and objectives.

The Advantage: "Self-healing."
Agents adapt to UI changes in real-time (like a new checkout step) without failing.

The Reality: Reduces maintenance drastically and allows QA to focus on high-level test strategy rather than fixing broken scripts.

The Power of Self-Healing



The Problem

Hardcoding: Traditional automation relies on specific page identifiers (CSS selectors, exact button placements).

The "False Failure": If developers add a new feature or change a button's color/location, the test fails—*not* because the application is broken, but because the script is lost.

The Maintenance Drain: QA engineers are forced to continuously rewrite code just to keep existing tests running.



The Solution

Contextual Understanding: AI agents evaluate the objective (e.g., "complete a checkout") rather than blindly following a rigid click-path.

Real-Time Adaptation: If a new step or button is introduced, the agent assesses the change, interacts with it appropriately, and continues the flow.

The Result: Tests "heal" themselves on the fly, drastically reducing false failures and freeing up engineering capacity.



Real world example: *A company completely changes its login provider to a new third-party system. A traditional script would immediately fail. The AI agent observes the new UI, figures out the new username/password fields, logs in, and continues the test suite transparently.*

Prerequisites for Adoption



Infrastructure

Isolated Staging Environment:

The AI agent needs a reachable, stable environment to interact with the application.

Core Requirement: If your workflow is just "develop locally and push straight to production," autonomous testing will struggle. You need a dedicated deployment URL for the agent to explore.



Technical Fit

Browser-Based Applications:

Currently, the most mature autonomous tools operate through web browsers.

The Boundary: If you are looking to test purely backend APIs or strictly native mobile applications, you may need specialized tools rather than a general browser-based AI agent.



A defined problem

Scaling Challenges: Your manual QA team cannot keep up with development speed (e.g., maintaining testing capacity despite team size changes).

Maintenance Burden: You have a massive suite of automated scripts, but your engineers are spending too much time fixing flaky tests instead of writing new code.

PART 2 OF 3

Under the Hood: AI Agents in Action

The "How" — Architecture, Real Examples & Limitations

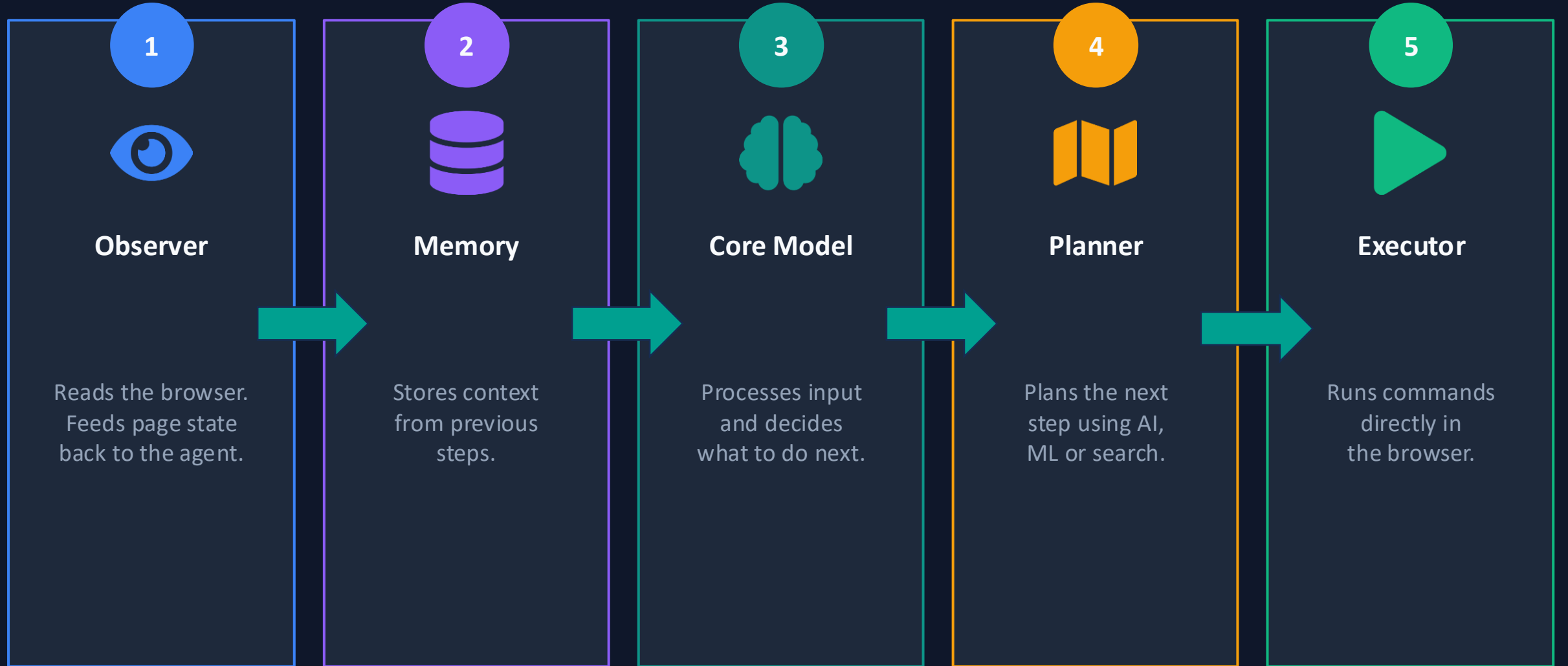
5 Core Components

Real-World Examples

Challenges & Limits

The 5 Components of an Autonomous Testing Agent

Each component feeds into the next — forming a continuous loop



00:15:41

↪ Continuous loop — after each action the Observer reads the browser again and the cycle restarts

AI Agents Adapting in the Wild

Two real cases where the agent recovered without any human intervention

Case 1 · The Login Provider Switch

BEFORE

Username & password login hardcoded by the customer

CHANGE

Customer replaced it with a third-party provider

RESULT

Agents ran transparently — zero problems, zero manual updates

"The agents handled the completely new login flow transparently — without any problems at all."

Case 2 · The Wiped Database

CONTEXT

E-commerce warehouse system. Test should configure a warehouse.

PROBLEM

Database was reset — no data existed for the test to run on.

RESULT

Agent created a new warehouse from scratch, then configured it.

"Instead of failing, the agent created a new warehouse and concluded the configuration still works."

Challenges & Current Limitations



False Positives & Negatives

Agents can pass broken features or flag working ones. Humans review results and feed corrections back so the agent learns via reflection or fine-tuning.



Complex Flows

Long, multi-system flows with domain knowledge are hard. If a human can't explain it easily, the agent will struggle even more.



Sensitive Data

Some clients had to abandon the tool because their staging environment had regulated data they couldn't share with an external tool.



Training Gap (Improving)

Early LLMs weren't trained on browser UIs. Anthropic Computer Use & OpenAI Operator are now closing this gap fast.

PART 3 OF 3

Implementation and Metrics

The Evolving Role of QA

The "Impact" — Adoption hurdles, key metrics & the semi-autonomous reality

Implementation & Hurdles

Key Metrics for Success

Semi-Autonomous Reality

Implementation & Hurdles

Getting started - Navigating the technical and organizational blockers



Getting Started

- Start small
- Repetitive, time-consuming tasks
 - e.g. smoke tests
- Addition to standard tools



Technical Blockers

- Having no staging environment
- Strict authentications
- Data privacy



Organizational Blockers

- Starting from scratch
- Fear of replacement
- Mindset: AI as superpower

Key Metrics for Success

Tracking the true impact of autonomous testing



Test Execution Time

What: Time of running entire test suite

Goal: Removing manual bottlenecks
-> shorter testing cycles



Ratio of Flaky Tests

What: Reduction of intermittent, false test failures

How AI helps: Visual Cues instead of failing during errors



Bug-Detection Rate

“The holy grail” for any QA team:
Measuring the actual number of real bugs the autonomous system discovers and prevents

The Evolving Role of QA

Embracing the semi-autonomous reality and new responsibilities



QA Manager

- **Role Shift:** from testers to strategic QA Managers
- **New Focus:** from maintaining broken scripts to more:
 - ❖ test planning
 - ❖ explorative testing
- **Collaboration:** Working closely with development teams
- + **Quality** across entire development cycle



Semi Autonomy

- **Current State:** True autonomy is not here yet
 - AI does repetitive tasks, but cannot replace the human
- **Human Oversight:** Testers still essential
 - ❖ Define goals
 - ❖ Understand context
 - ❖ Oversee AI actions
- **Continuous Upskilling:** Rather than new skills -> curiosity and learn AI tools

Thank You

Questions & Discussion

Donato Giuliano - UO318987

Marvin Kickel - UO318535

Daniel Onís Fabián – UO293946